

// SETUP GUIDE

Agents Workshop

Smart Agent in 10 minutes

Git

Ollama

uv

Streamlit

MLflow

8 steps · about 15 minutes

STEP 1

Install Git

macOS

```
$ brew install git
```

Windows

```
git-scm.com/downloads
```

Linux

```
$ sudo apt install git
```

STEP 2

Clone the Repository

```
$ git clone https://github.com/mohamedadaly/agents-workshop.git  
$ cd agents-workshop
```

STEP 3

Install an Editor (or Skip It)

VS Code (recommended)

Download from code.visualstudio.com

```
$ code .
```

Terminal only

No editor required — every step here runs from a terminal.

STEP 4

Install Ollama & Pull a Model

Download

```
ollama.com/download
```

Pull the model

```
$ ollama pull qwen3.5:4b-mlx
```

Make sure Ollama is running — it starts automatically after install, or run **ollama serve**.

STEP 5

Install uv

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows: see docs.astral.sh/uv/getting-started/installation

STEP 6

Install Project Dependencies

```
$ uv sync
```

Reads **pyproject.toml** / **uv.lock** and creates a local **.venv** with everything installed:

streamlit

deepagents

langchain-ollama

langchain-community

mlflow

STEP 7

Run the Streamlit App

```
$ uv run streamlit run 01_deep_agent.py
```

Opens in your browser. In the sidebar: pick a model, confirm the database URI (defaults to **Chinook.db**), then click **Connect** before chatting.

STEP 8 · OPTIONAL

View MLflow Traces

Every question you ask is traced automatically. In a separate terminal:

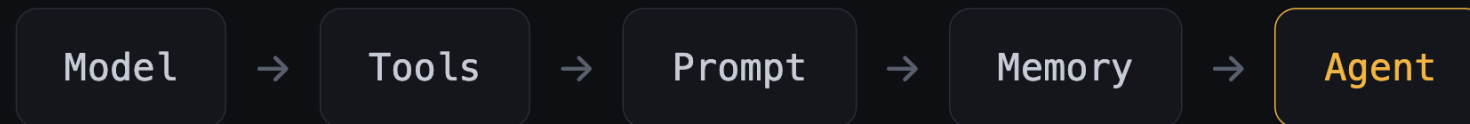
```
$ uv run mlflow ui --backend-store-uri sqlite:///mlflow.db
```

Then open the URL it prints — typically **http://localhost:5000**

// CODE WALKTHROUGH

Anatomy of the Agent

Four pieces, wired together in **build_agent()**



A Local Ollama Model

```
llm = ChatOllama(  
    model=model,  
    base_url=base_url,  
    validate_model_on_init=True,  
    temperature=0,  
)
```

Picked in the sidebar at connect time. **temperature=0** keeps SQL generation deterministic; **validate_model_on_init** fails fast if the model isn't pulled yet.

Database & Tools

```
db = SQLiteDatabase.from_uri(db_uri)
tools = SQLiteDatabaseToolkit(db=db, llm=llm).get_tools()
```

The toolkit hands the agent four ready-made LangChain tools:

sql_db_list_tables

sql_db_schema

sql_db_query_checker

sql_db_query

Steering the Agent

Passed as **system_prompt=INSTRUCTIONS** — four rules keep the ReAct loop safe and on-task:

Look before you query

Verify table names with **sql_db_list_tables**, inspect schemas with **sql_db_schema** first.

Check before you run

Run non-trivial queries through **sql_db_query_checker** before **sql_db_query**.

Read-only, always

No INSERT, UPDATE, DELETE, DROP — ever.

Plan multi-step work

Use the agent's built-in todo list before a multi-step question.

The Prompt, Verbatim

You are an agent designed to interact with a SQL database.

Given an input question, create a syntactically correct sqlite query, run it, inspect the results, and return a concise answer. Unless the user asks for a specific number of rows, limit results to at most 10. Never **SELECT *** – only request the columns you need.

Do NOT issue any **DML** statements (INSERT, UPDATE, DELETE, DROP, etc.).

Always verify table names with **sql_db_list_tables** and inspect schemas with **sql_db_schema** before writing a query. Use **sql_db_query_checker** on any non-trivial query before running it with **sql_db_query**.

For multi-step questions, use your todo list to plan the steps before you start querying.

Answer in plain language. Do not describe the database schema or the steps you took.

The exact **INSTRUCTIONS** string from **01_deep_agent.py** — the four rules on the previous slide, in the agent's own words.

Memory via Checkpointer

```
checkpointer=InMemorySaver()  
  
run_config = {"configurable": {"thread_id": thread_id}}  
agent.invoke({"messages": [...]}, config=run_config)
```

Same **thread_id** on every turn — the checkpointer replays the full message history, so the agent remembers earlier questions in the conversation.

create_deep_agent()

```
agent = create_deep_agent(  
    model=llm,  
    tools=tools,  
    system_prompt=INSTRUCTIONS,  
    checkpointer=InMemorySaver(),  
)
```

create_deep_agent wraps all four pieces in a LangGraph ReAct loop, with planning and a virtual file system built in — this agent only ever reaches for the four SQL tools above.

// WHAT POWERS THIS AGENT

ReAct: Reasoning + Acting

The pattern behind `create_deep_agent`'s loop — and most LLM agents since 2022.



1 · ORIGIN

Why It Was Invented

Two 2022-era approaches each got half the job done:

Reasoning-only (Chain-of-Thought)

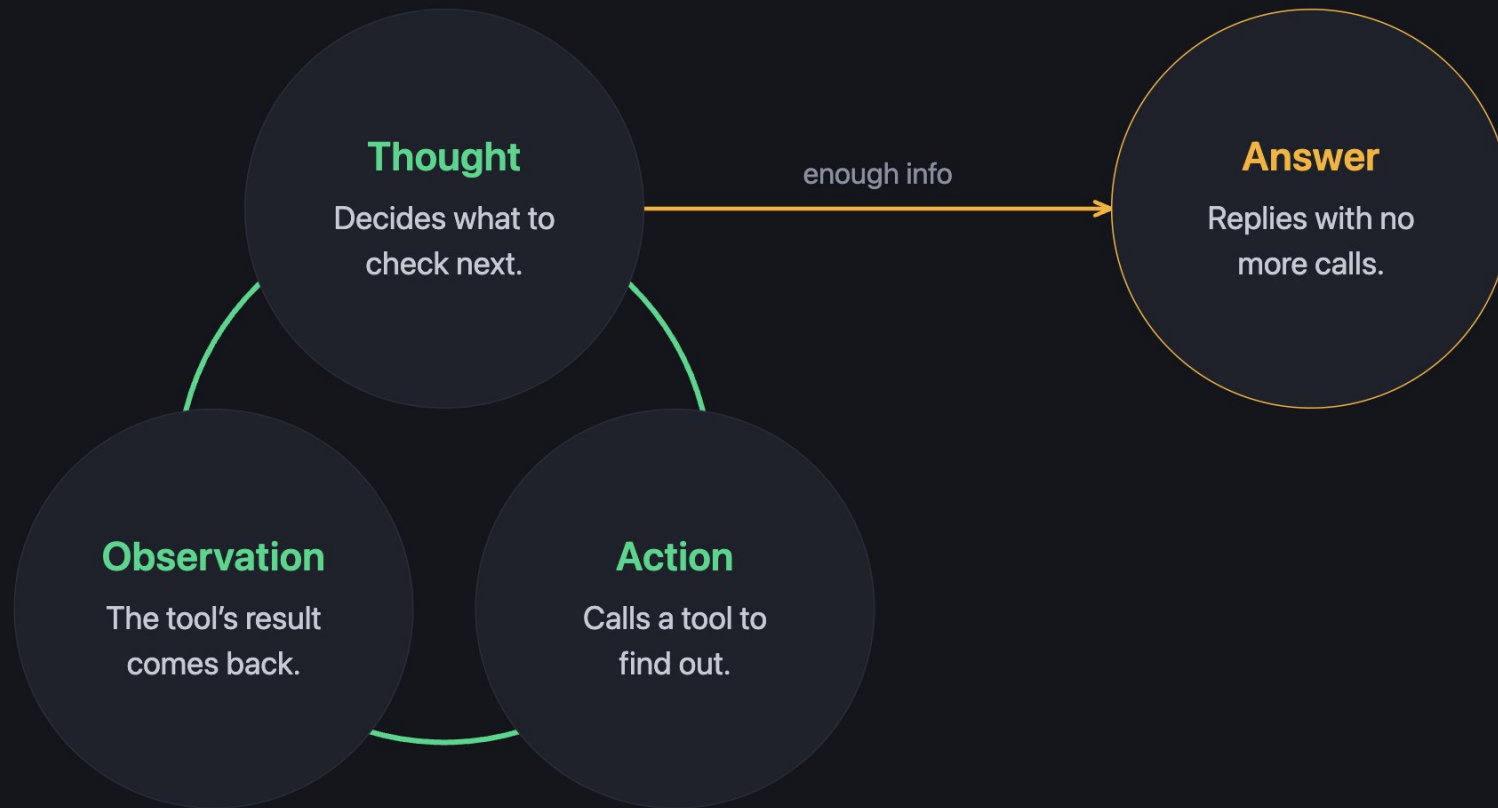
Great at step-by-step logic, but frozen in time — no way to look anything up, so it confidently hallucinates facts it can't know.

Acting-only agents

Can call tools, but act impulsively — no visible plan, hard to debug, and errors compound with no reflection.

ReAct (Yao et al., 2022) interleaves both: reasoning steers which action to take next, and each action's result corrects the reasoning.

How the Loop Works



Mapped Onto the Code

ReAct step	In this agent's trajectory
Thought	folded into the model's next tool call
Action	<code>AIMessage(tool_calls=[sql_db_schema(...)])</code>
Observation	<code>ToolMessage(content=<query result>)</code>
Repeat	LangGraph's model ↔ tools nodes run again
Answer	<code>AIMessage, no tool_calls → trajectory[-1].content</code>

Every row above is exactly what the **Agent trajectory** expander in the app shows you, turn by turn.

Why It Matters

Grounded

The agent checks the real schema instead of guessing a column name — fewer hallucinated facts.

Interpretable

Every thought, action, and observation is a message you can read — literally the trajectory expander.

Recoverable

A bad query doesn't fail silently — the next loop iteration can look again and correct course.

Extensible

Deep agents layer planning and sub-agents on top of this same Thought → Action → Observation core.